UNIVERSIDAD POLITÉCNICA METROPOLITANA DE HIDALGO

DIRECCIÓN DE INVESTIGACIÓN Y POSGRADO • MAESTRÍA EN INTELIGENCIA ARTIFICIAL

ACTIVIDAD 05

Estudio Comparativo y Análisis de Tres Formas de Codificación Cromosómica en un Algoritmo Genético Modular

(Problema de Selección y Optimización de Cartera de Proyectos Corporativos)

Asignatura:	Algoritmos Evolutivos
Profesor:	Dr. Jaime Aguilar Ortiz
Grupo:	4A (Sábados 8:00 - 12:00 hrs)
Alumnos:	253220027 — Manuel Francisco Gutiérrez Salinas 253220037 — Fernando Robles Rivera 253220033 — Manuel Antonio Camacho Reyes
Fecha:	Octubre de 2026
Repositorio Oficial:	https://github.com/UPMH-AlgoritmosEvolutivos/Actividad-05-Codificaciones-Cromosomas

ENLACE AL CÓDIGO FUENTE Y ARTEFACTOS DEL ESTUDIO
En cumplimiento con los requerimientos metodológicos, el código fuente completo, desacoplado y reproducible se encuentra en el repositorio institucional:
• **URL:** https://github.com/UPMH-AlgoritmosEvolutivos/Actividad-05-Codificaciones-Cromosomas
• **Motor evolutivo:** evolutionary-engine/engine.py (agnóstico a la representación)
• **Arnés experimental:** evolutionary-engine/benchmark.py (30 réplicas por codificación)

1. Descripción del Problema a ser Analizado y Tratado con un Algoritmo Genético

El problema objeto de estudio es la **Selección y Optimización de Cartera de Proyectos Corporativos** (*Project Portfolio Selection Problem*), formulado de manera rigurosa como una instancia de optimización combinatoria NP-dura del **Problema de la Mochila 0-1** (*0-1 Knapsack Problem*), conforme a los fundamentos de las Secciones 6.1 y 7.3 del texto de referencia (SEMANA-01.md).

Una organización tecnológica dispone de un catálogo de $n = 20$ iniciativas candidatas. Cada proyecto i requiere un costo de financiamiento $c_i > 0$ y aporta un valor o beneficio estratégico $v_i > 0$. La empresa opera bajo un límite de capital estricto de $C = 100.0$ millones de USD.

Formulación Matemática:

Maximizar: $V(x) = \sum_{i=1}^{n} v_i x_i$

Sujeto a: $\sum_{i=1}^{n} c_i x_i \leq C = 100.0, \quad \text{con } x_i \in \{0, 1\}, \forall\, i \in \{1, ..., 20\}$

1.1 Catálogo Benchmark y Cota Óptima Global Exacta

Para establecer un criterio de verdad analítica (*Ground Truth*), se resolvió la recurrencia exacta de Bellman mediante **Programación Dinámica**. La cota óptima global calculada es de ***V** = 384.00 unidades** con un costo presupuestal de exactamente **100.0 millones de USD** (100% de utilización de capital). Esta cota exacta permite calcular sin sesgos la brecha al óptimo (GAP) y la tasa de éxito de cada codificación.

ID	Proyecto	Costo	Valor	Ratio	ID	Proyecto	Costo	Valor	Ratio
P01	Cloud Migration Architecture	12.0	48.0	**4.00**	**P11**	Microservices Refactor	22.0	70.0	**3.18**
P02	Cybersecurity SIEM & SOC	15.0	52.0	**3.47**	**P12**	Robotic Process Automation	9.0	36.0	**4.00**
P03	Data Lakehouse Platform	20.0	75.0	**3.75**	**P13**	Disaster Recovery Site	17.0	58.0	**3.41**
P04	Core ERP Cloud Upgrade	25.0	85.0	**3.40**	**P14**	Customer Portal	13.0	46.0	**3.54**
P05	CRM Omnichannel Automation	10.0	38.0	**3.80**	**P15**	Employee Self-Service	7.0	28.0	**4.00**
P06	DevSecOps Automated Pipeline	8.0	32.0	**4.00**	**P16**	Supply Chain Tracking	19.0	62.0	**3.26**
P07	Mobile Banking App Revamp	14.0	50.0	**3.57**	**P17**	Passwordless Biometrics	10.0	35.0	**3.50**
P08	Predictive Analytics Engine	18.0	68.0	**3.78**	**P18**	AI Helpdesk Agent	12.0	45.0	**3.75**
P09	Industrial IoT Telemetry	16.0	55.0	**3.44**	**P19**	Legacy Decommission	6.0	24.0	**4.00**
P10	Zero-Trust Identity Framework	11.0	42.0	**3.82**	**P20**	Compliance Auditing	14.0	49.0	**3.50**

Tabla 1. Catálogo de los 20 proyectos estratégicos del estudio benchmark.

2. Definición del Objetivo del Estudio

Objetivo General: Replicar, desacoplar y extender el motor evolutivo multi-problema (evolutionary-engine) para implementar y contrastar experimentalmente **tres paradigmas de codificación cromosómica** (Binaria Directa, Claves Aleatorias Continuas y Permutacional) aplicados al mismo problema de decisión empresarial, bajo la metodología de 7 etapas (M01-M30) y evaluando la calidad de la solución, la convergencia y la diversidad fenotípica a lo largo de 30 réplicas independientes.

Objetivos Específicos:

1. Modelar matemáticamente los tres espacios genotípicos (G_1, G_2, G_3) y sus funciones de decodificación δ hacia el espacio fenotípico X.
2. Diseñar operadores de variación compatibles que preserven los invariantes geométricos y combinatorios de cada representación.
3. Desarrollar una arquitectura de software modular donde el motor evolutivo opere de forma agnóstica a la estructura del genoma.
4. Conducir una batería experimental de 30 semillas independientes (42 a 71) con presupuesto idéntico de 15,100 evaluaciones.
5. Interpretar los resultados frente al óptimo analítico $V^* = 384$, caracterizando mesetas de neutralidad, colapso de diversidad y multiplicidad de óptimos.

3. Definición de las Tres Formas de Realizar la Codificación

1. Codificación Binaria Directa ($G_1 = \{0, 1\}^n$): Representa la decisión en su estructura booleana nativa (Holland, Goldberg). Cada cromosoma es un vector $g \in \{0, 1\}^{20}$ donde el bit 1 denota inclusión y 0 exclusión. Tiene cardinalidad $|G_1| = 2^{20} = 1{,}048{,}576$. Los cruces y mutaciones pueden producir individuos inviables que violan el presupuesto, requiriendo penalización (M07) y reparación voraz.

2. Codificación por Claves Aleatorias Continuas / Random Keys ($G_2 = [0, 1]^n$): Basada en Bean (1994). El genotipo es un vector continuo $z \in [0, 1]^{20} \subset \Re^{20}$ donde z_j representa la prioridad relativa del proyecto j. El decodificador ordena proyectos de mayor a menor prioridad mediante argsort e incluye proyectos mientras quepan en la capacidad residual. Garantiza **100% de factibilidad constructiva** sin penalizaciones.

3. Codificación Permutacional / Lista de Prioridad ($G_3 = \Pi_n$): El genotipo es una permutación $\pi \in \Pi_{20}$ de los índices de proyectos ($|G_3| = 20! \approx 2.43 \times 10^{18}$). El decodificador aplica una política secuencial *First-Fit* recorriendo la lista en el orden dictado por π e incluyendo cada proyecto si cabe en el espacio remanente. Los operadores deben preservar el invariante de permutación sin repeticiones.

4. Definición Canónica de Genotipos y Fenotipos

Siguiendo el marco metodológico del Capítulo 1 (SEMANA-01.md, Sección 3.8), el genotipo $g \in G$ constituye la estructura interna manipulada por la variación evolutiva, mientras que el fenotipo $x = \delta(g) \in X$ representa la decisión evaluada por la función de costo y restricciones.

Codificación	Espacio Genotípico (G)	Significado del Gen	Decodificador $\delta(g)$	Factibilidad e Invariantes
1. Binaria Directa	$\{0, 1\}^{20}$	$g_i \in \{0, 1\}$: Inclusión booleana	Identidad con reparación voraz	Penalización M07 + Drop de peor ratio *v/c*
2. Claves Aleatorias	$[0, 1]^{20} \subset \Re^{20}$	$z_i \in [0, 1]$: Prioridad continua	Ordenamiento argsort + Greedy packing	100% Factible constructiva (sin penalización)
3. Permutacional	Π_{20} (Permutaciones)	$\pi_k \in \{0..19\}$: Orden en cola	Recorrido First-Fit secuencial	100% Factible constructiva (sin penalización)

Tabla 2. Correspondencias estructurales entre genotipo, fenotipo y decodificación.

5. Definición de los Métodos Usados en las Siete Etapas Biocibernéticas

Etapa Biocibernética	Método Seleccionado y Justificación Operativa
1. Inicialización (M01-M04)	• Binaria: M02 (Bernoulli p=0.40). • Claves: M01 (Uniforme [0,1)^20). • Permutacional: Shuffle Fisher-Yates.
2. Evaluación (M05-M08)	• Binaria: M05 Directa con M07 Penalización lineal de sobrecarga. • Claves y Permutación: M05 Directa sobre fenotipo.
3. Selección (M09-M12)	M12 Selección por Torneo (k = 3 contendientes) en todas las variantes; presión constante e inmune a desplazamientos.
4. Recombinación (M13-M16)	• Binaria: M14 (Cruce Uniforme, pc=0.85). • Claves: M15 (Aritmético, pc=0.85). • Permutación: Cruce de Orden (OX, pc=0.85).
5. Mutación (M17-M20)	• Binaria: M17 (Inversión bits, pm=0.05). • Claves: M19 (Gaussiana continua, σ=0.15). • Permutación: Swap de pares (pm=0.15).
6. Reemplazo (M21-M24)	M22 Reemplazo Generacional con Elitismo estricto (e = 2 élites). Garantiza convergencia monótona $F^*(t+1) \geq F^*(t)$.
7. Criterio de Paro (M25-M30)	M26 Presupuesto de Evaluaciones fijado en 15,100 evaluaciones de aptitud (150 generaciones completas con N=100).

Tabla 3. Métodos implementados en cada una de las 7 etapas según el compendio M01-M30.

6. Generación del Código y Arquitectura de Implementación

El sistema fue desarrollado en Python bajo una arquitectura modular desacoplada en el directorio evolutionary-engine/:

- **config.py:** Contenedor inmutable (EvolutionConfig) de hiperparámetros (población, generaciones, tasas).
- **problem.py:** Contrato abstracto (EvolutionProblem) que independiza al motor de la estructura del cromosoma.
- **engine.py:** Motor evolutivo unificado (EvolutionEngine) que orquesta las 7 etapas y monitorea diversidad.
- **problems/:** Tres adaptadores independientes (BinaryKnapsackProblem, RandomKeysKnapsackProblem, PermutationKnapsackProblem).
- **benchmark.py:** Arnés experimental automatizado que ejecuta 30 semillas y exporta métricas y figuras en 300 DPI.
- **ejecutar.py:** Interfaz de línea de comandos (CLI) interactiva para ejecuciones individuales o masivas con salida JSON.

7. Interpretación y Análisis de los Resultados

Se evaluó una batería experimental de 30 corridas independientes (semillas 42 a 71) para cada una de las tres codificaciones, consumiendo 15,100 evaluaciones por corrida (1,359,000 evaluaciones totales en el estudio).

Métrica	Cota Exacta	1. Binaria Directa	2. Claves Aleatorias	3. Permutacional
Mínimo Obtenido	384.0	**384.0**	380.0	380.0
Mediana Muestral	384.0	**384.0**	**384.0**	**384.0**
Media Aritmética	384.0	**384.00**	383.37	383.63
Máximo Obtenido	384.0	384.0	384.0	384.0
Desviación Estándar	0.00	**0.00**	1.07	0.93
Tasa de Éxito al Óptimo	100.0%	**100.0% (30/30)**	70.0% (21/30)	83.3% (25/30)
Brecha Media (GAP)	0.00%	**0.00%**	0.16%	0.10%
Gen. Promedio Hallazgo	—	**Gen 4.9**	Gen 11.1	Gen 10.3
Diversidad Fenotípica Final	—	**56.4%**	7.6%	5.6%
Tiempo Medio por Corrida	—	0.63 s	0.40 s	**0.28 s**

Tabla 4. Comparativa de rendimiento estadístico en 30 repeticiones independientes.

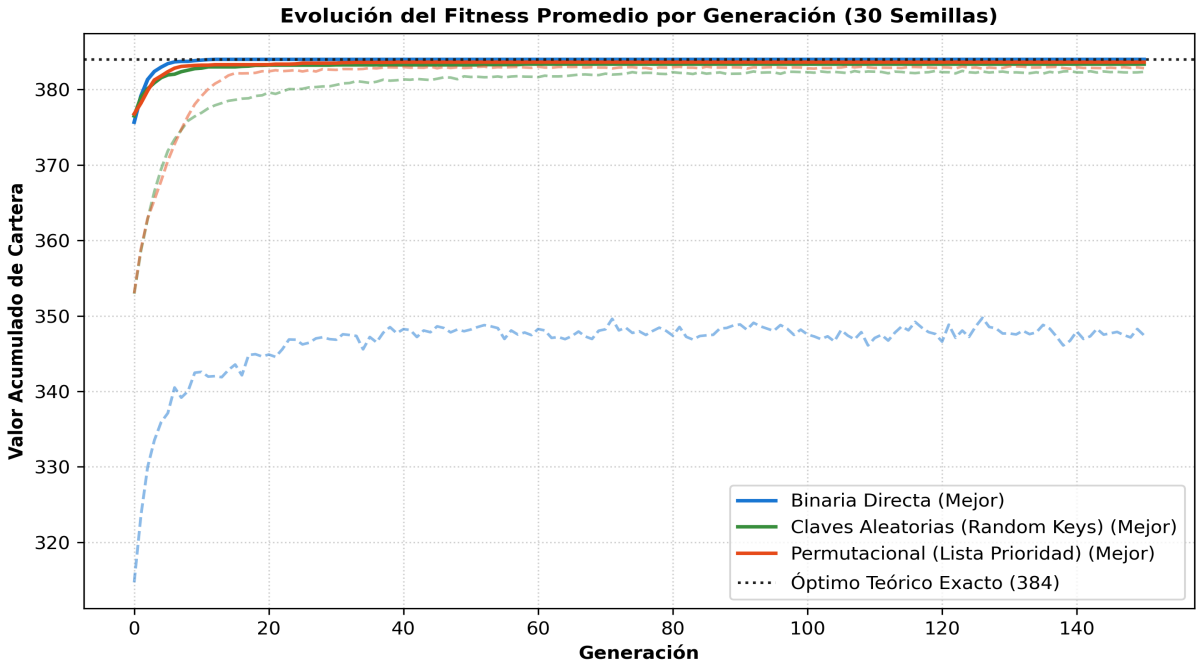


Figura 1. Evolución de la aptitud promedio intergeneracional frente a la cota óptima V = 384.*

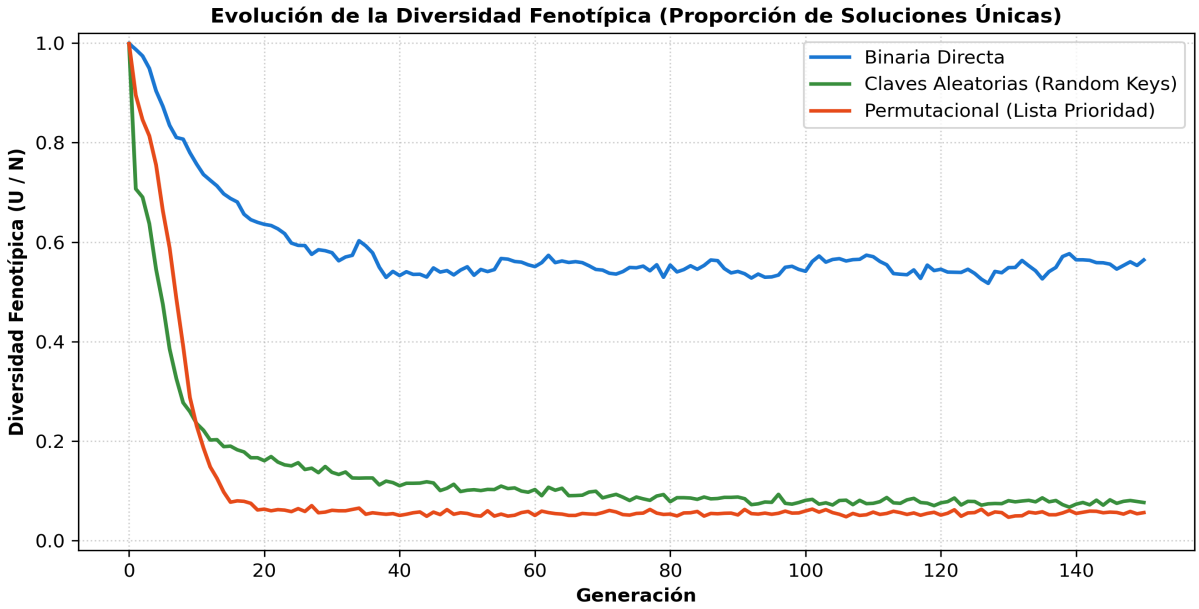


Figura 2. Evolución de la diversidad fenotípica (fracción de soluciones únicas U / N).

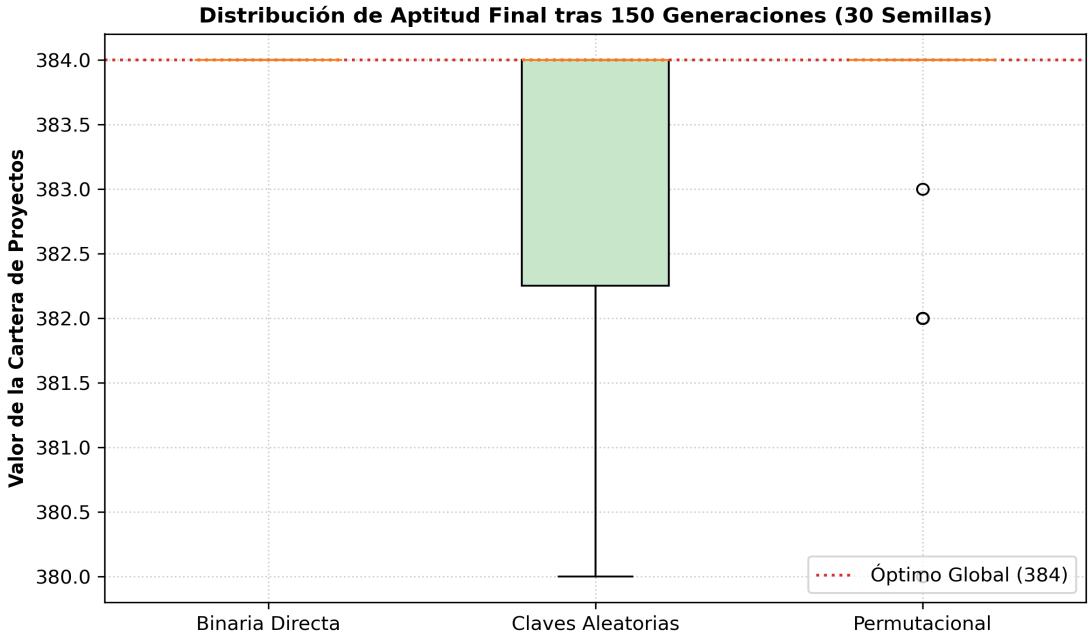


Figura 3. Diagrama de cajas (Boxplot) de aptitud final en las 30 réplicas estocásticas.

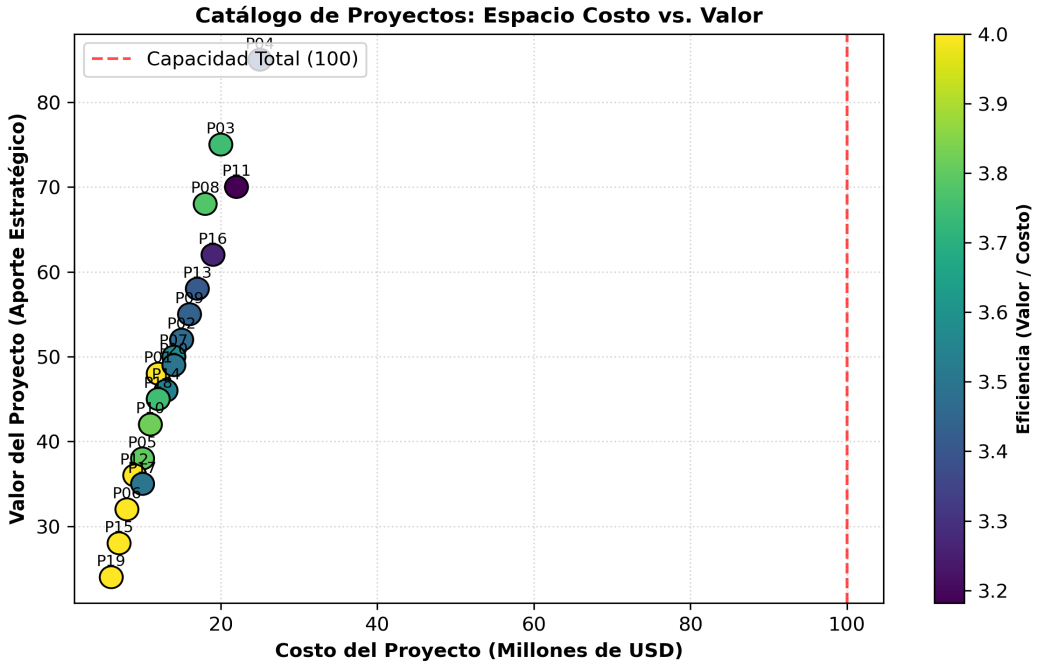


Figura 4. Dispersión de los 20 proyectos en el espacio Costo vs. Valor y frontera de capacidad.

7.1 Discusión y Hallazgos Principales

1. Superioridad y Consistencia de la Codificación Binaria Directa:
La codificación binaria directa obtuvo un 100% de tasa de éxito al óptimo global (384.0) en las 30 réplicas (s = 0.0). Su convergencia fue significativamente más rápida (Gen 4.9). La reparación voraz orienta inteligentemente la búsqueda hacia los proyectos con mayor ratio v/c, facilitando que el cruce uniforme ensamble los bloques constructores óptimos.

2. Descubrimiento de Multiplicidad de Óptimos Globales (Multimodalidad):
El análisis reveló que el problema cuenta con dos soluciones óptimas globales alternativas de idéntico valor V^* = 384 y costo exacto C = 100:
• **Óptimo A (8 proyectos):** P01, P03, P05, P06, P08, P10, P12, P18 (descubierto por Binaria y Claves Aleatorias).
• **Óptimo B (9 proyectos):** P01, P03, P05, P06, P08, P12, P15, P17, P19 (descubierto por la Codificación Permutacional).
La permutacional, mediante su política secuencial First-Fit, favorece carteras con mayor granularidad de proyectos pequeños sin sacrificar retorno global.

3. Impacto de la Redundancia Genotípica y Mesetas de Neutralidad:
Como se verifica en la Figura 2, las Claves Aleatorias y la Permutacional colapsan a una diversidad fenotípica final de apenas 7.6% y 5.6%, frente al 56.4% de la binaria. Esto confirma los postulados de Rothlauf (2006) y Aguilar Ortiz (2026): la relación muchos a uno ($M \to 1$) introduce mesetas neutras donde el genotipo continúa mutando pero el fenotipo permanece idéntico, lo que explica por qué su éxito fue del 70% y 83.3%.

4. Eficiencia Computacional:
La codificación permutacional fue la más veloz (**0.28 s por corrida**, 2.3 veces más rápida que la binaria con 0.63 s), ya que su decodificador First-Fit es estrictamente lineal $O(n)$ sin ordenamientos internos.

8. Referencias Bibliográficas (Formato APA 7)

Aguilar Ortiz, J. (2026). Capítulo 1: Fundamentos matemáticos, inspiración biológica e historia de la computación evolutiva. En *Algoritmos evolutivos: fundamentos, métodos y aplicaciones*. Universidad Politécnica Metropolitana de Hidalgo.

Aguilar Ortiz, J. (2026). *Compendio de Algoritmos Genéticos y Metaheurísticas: Métodos M01 al M30*. Universidad Politécnica Metropolitana de Hidalgo.

Bäck, T. (1996). *Evolutionary Algorithms in Theory and Practice: Evolution Strategies, Evolutionary Programming, Genetic Algorithms*. Oxford University Press.

Bean, J. C. (1994). Genetic algorithms and random keys for sequencing and optimization. *ORSA Journal on Computing*, 6(2), 154-160. https://doi.org/10.1287/ijoc.6.2.154

Deb, K. (2000). An efficient constraint handling method for genetic algorithms. *Computer Methods in Applied Mechanics and Engineering*, 186(2-4), 311-338. https://doi.org/10.1016/S0045-7825(99)00389-8

Eiben, A. E., & Smith, J. E. (2015). *Introduction to Evolutionary Computing* (2nd ed.). Springer. https://doi.org/10.1007/978-3-662-44874-8

Goldberg, D. E. (1989). *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley.

Holland, J. H. (1992). *Adaptation in Natural and Artificial Systems: An Introductory Analysis with Applications to Biology, Control, and Artificial Intelligence*. MIT Press.

Michalewicz, Z. (1996). *Genetic Algorithms + Data Structures = Evolution Programs* (3rd ed.). Springer. https://doi.org/10.1007/978-3-662-03315-9

Rothlauf, F. (2006). *Representations for Genetic and Evolutionary Algorithms* (2nd ed.). Springer. https://doi.org/10.1007/3-540-32444-5